

初识Lambda

北京理工大学计算机学院
金旭亮



概述



- Lambda是Java 8引入的一个新特性，它也是使用Java进行“函数式编程”的主要技术手段。
- 在开发中使用Lambda，能帮助我们写出简洁易懂的代码，有效地提升代码的可维护性，并且使得开发者在Java项目中可以大规模地应用函数式编程范式。
- 时至今日，有大量的Java技术使用了Lambda，可以说，Lambda在Java开发中“无处不在”！
- 下面，我们通过一些例子来帮助同学们理解和掌握Lambda表达式。

预热：了解SAM接口的概念



只定义有一个抽象方法的接口，称为“**单一抽象方法 (SAM: Single Abstract Method)**”的接口，在开发中可以有三种方式实现它：

顶层类

内部类

匿名类

下面我们通过代码来展示这些语法特性.....

接口的定义与传统实现方式



先来看以下“常规”代码段：

```
interface MyInterface {  
    void func();  
}  
  
class MyClass implements MyInterface {  
    @Override  
    public void func() {  
        System.out.println("MyClass's func()");  
    }  
}
```

上述代码定义了一个MyInterface接口（它只定义了一个抽象方法，所以是SAM接口），接着，写了一个MyClass类实现这个接口，在这里，MyClass是一个顶层类。

接口的典型使用方法



然后，写了一个静态方法调用接口定义的方法

```
public static void doWithMyInterface(MyInterface obj) {  
    obj.func();  
}
```

使用传统编程方式，上面的代码是这样被调用的：

```
//传统方法，定义一个类实现接口，创建这个类的对象，  
//再把它传给doWithMyInterface()方法  
MyClass obj = new MyClass();  
doWithMyInterface(obj);
```

参看示例：IntroduceLambda.java

使用本地内部类实现接口



除了使用顶层类，也可以使用内部类实现接口：

```
//或者写一个本地内部类，实现这一接口
class MyInnerClass implements MyInterface{
    @Override
    public void func() {
        System.out.println("本地内部类，实现接口");
    }
}

//实例化本地内部类对象，传给示例方法
doWithMyInterface(new MyInnerClass());
```

注意一下MyInnerClass是定义在一个方法（示例中名为traditionalCode）内部的。



内部类的适用场景，主要是“仅在本类内部使用”，不需要被外界调用，并且代码比较简短。

使用匿名内部类实现接口



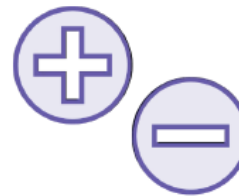
特别地，如果代码仅在特定方法内部调用，并且代码量也不大，可以直接使用匿名内部类实现接口：

```
doWithMyInterface(new MyInterface() {  
    @Override  
    public void func() {  
        System.out.println("使用匿名内部类，实现接口");  
    }  
});
```



在Java 8之前，这种方式被大量的使用，尤其是在有可视化界面的桌面应用中，它被用来给各种控件的事件添加事件响应代码。

“传统”编码方式的弱点



- 前面的代码是传统的经典的Java面向对象编程代码，在开发中使用接口，所有代码可以很明确地分为“第一步、第二步、第三步……”，很易于理解，也很规范，但每次都需要手工做这么多的事，似乎有点过于麻烦了。
- 能不能简化一下，提高编程效率？

为达到这个目的，Java设计者借鉴其他编程语言，把Lambda特性引入到了Java中。

知新：Lambda表达式长得什么样？



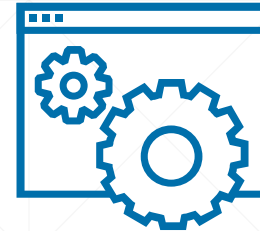
使用Java 8所引入的Lambda特性，前述代码可以改写为：

```
//Java 8: 定义一个Lambda表达式，将其引用保存到变量中，  
//再把它传给doWithMyInterface()方法  
//从而可以节省下新定义一个类的工作任务  
MyInterface lambdaObj = () -> {  
    System.out.println("Explicit Define Lambda object's func()");  
};  
doWithMyInterface(lambdaObj);
```

可以更进一步地简化为：

```
//Java 8: 直接把一个Lambda表达式作为doWithMyInterface()方法的参数  
//不仅不需要定义一个单独的类，甚至不再需要定义一个变量  
doWithMyInterface(() -> {  
    System.out.println("inline lambda object's func()");  
});
```

直接执行Lambda表达式



要“执行”一个Lambda表达式所封装的代码，需要使用关联接口所定义的方法：

```
interface MyInterface {  
    void func();  
}
```

```
MyInterface lambdaObj2 = () -> {  
    System.out.println("另一个Lambda表达式");  
};  
//Lambda表达式，也可以直接执行  
lambdaObj2.func();
```



Lambda表达式的类型，由接收它的SAM接口确定。它的执行，可以使用SAM接口所定义的“唯一”公有方法。



通过前面这个例子，我们可以得到以下的这个印象：

使用Lambda和不用Lambda，代码都能完成相同的功能，使用Lambda的好处可以用两个字来概括——

简洁

什么是Lambda表达式?

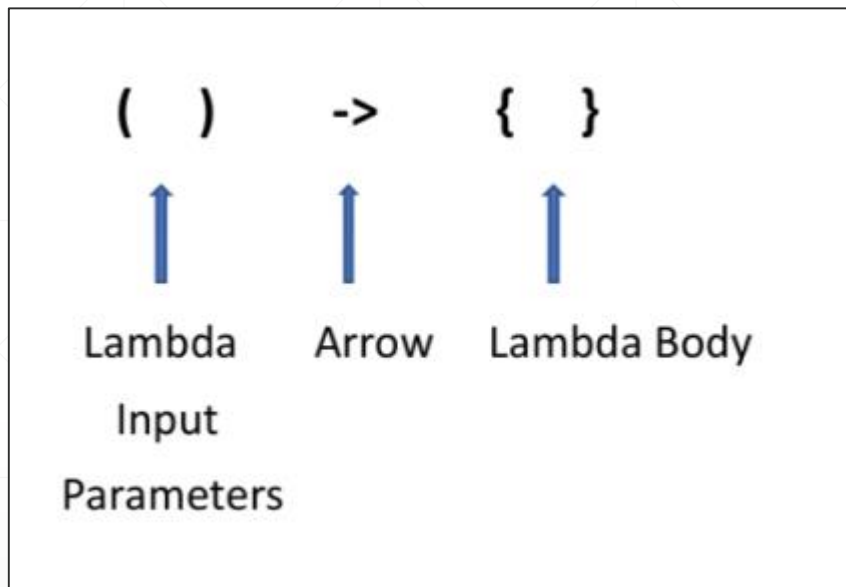
一个Lambda表达式，可以看成是一小段代码，这段代码你可以把它当作整体看待，把它传来传去，并且在需要时可以随时执行它，比如前面示例中的Lambda表达式，就成为了另一个函数调用时的“实参”：

```
//Java 8: 直接把一个Lambda表达式作为doWithMyInterface()方法的参数
//不仅不需要定义一个单独的类，甚至不再需要定义一个变量
doWithMyInterface(() -> {
    System.out.println("inline lambda object's func()");
});
```



Lambda特性的关键点之一是“**延迟执行 (deferred execution)**”，仅定义一个Lambda表达式变量，并不意味着它会马上执行，只有显式调用它，它才会执行。

定义Lambda表达式的语法



如果只有一个参数，圆括号可以省去。

如果只有一条语句，大括号可以省去。

理解Lambda



可以把 Lambda 表达式看成是一个匿名函数，它基本上就是一个没有声明名称的方法，但它却有和匿名类实例类似的功能——能作为参数传递给一个方法。

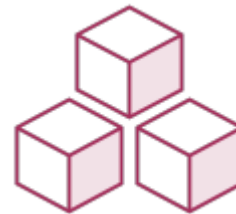


Lambda并没有增强Java语言的功能，它只是让写代码更为简洁和方便。



在Java 8之前做不了的事情，Lambda也做不了。

Lambda表达式用在什么地方？



1

赋值给一个函数式（SAM）接口变量

2

作为方法调用的实参（参数类型是SAM接口）

SAM接口类型的变量（或参数），是Lambda表达式的唯一接收者，SAM接口有另一个更常用的名字，叫作“函数式接口（Functional Interface）” 下一讲，我们就会来说说这个“函数式接口”

不得不说：Java Lambda的尴尬之处



由于没有引入真正的“函数类型”，Java的Lambda特性必须依托于特定的函数式接口，这就使得调用它变得比较麻烦，你必须使用函数式接口中所定义的抽象方法去调用一个接收了Lambda表达式的变量，才能执行这个Lambda表达式中的代码。



相比之下，C#定义了委托，Kotlin定义了函数类型，其变量都可以接收Lambda表达式，执行Lambda表达式，可以直接使用“函数变量(参数列表)”这种方式来完成，要简洁易懂得多。

